

BÀI 27: THAM SỐ CỦA HÀM

SAU BÀI NÀY EM SẼ

- Biết cách thiết lập các tham số của hàm.
- Hiểu được cách truyền giá trị thông qua đối số của hàm.
- Biết viết chương trình có sử dụng chương trình con (hàm).

KHỞI ĐỘNG

Quan sát các lệnh sau và cho biết sự khác nhau giữa **tham số** (parameter) và **đối số** (argument).

Python

'a', 'b', 'c' trong định nghĩa hàm là các tham số

```
def f_sum(a, b, c):  
    return a + b + c
```

Gọi hàm

x, y = 5, 3

10, x, y khi gọi hàm là các đối số

ket_qua = f_sum(10, x, y)

print(ket_qua) # Kết quả: 18

1. THAM SỐ VÀ ĐỐI SỐ CỦA HÀM

Hoạt động 1: Phân biệt tham số và đối số

Quan sát ví dụ sau để tìm hiểu cách dữ liệu được truyền vào hàm.

Python

Dòng 1: Định nghĩa hàm với các tham số a, b, c

```
>>> def f(a, b, c):
```

```
...     return a + b + c
```

Dòng 3: Gọi hàm với các đối số là giá trị cụ thể

```
>>> f(1, 2, 3)
```

```
6
```

Dòng 5: Gán giá trị cho các biến x, y, z

```
>>> x, y, z = 10, 20, 5
```

Dòng 6: Gọi hàm với các đối số là biến

```
>>> f(x, y, z)
```

```
35
```

Dòng 8: Lỗi gọi hàm bị lỗi

```
>>> f(a, b, c)
```

Traceback (most recent call last):

```
...
```

NameError: name 'a' is not defined

Giải thích:

- **Dòng 1:** Hàm f() được định nghĩa với ba **tham số** là a, b, c. Các tham số này được dùng như biến bên trong hàm.

- **Dòng 3 & 6:** Khi gọi hàm, các giá trị được truyền vào (như 1, 2, 3 hoặc các biến x, y, z) được gọi là **đối số**. Giá trị của các đối số sẽ được gán cho các tham số tương ứng.
- **Dòng 8:** Lỗi gọi hàm bị lỗi vì các biến a, b, c được dùng làm đối số nhưng chưa được gán giá trị ở bên ngoài hàm.

GHĨ NHỚ

- **Tham số (parameter)** được khai báo khi định nghĩa hàm và được dùng như biến bên trong hàm.
- **Đối số (argument)** là giá trị cụ thể (hoặc biến chứa giá trị) được truyền vào cho tham số khi gọi hàm.
- Số lượng đối số khi gọi hàm phải bằng số lượng tham số trong định nghĩa hàm.

2. CÁCH SỬ DỤNG CHƯƠNG TRÌNH CON

Hoạt động 2: Khi nào nên sử dụng chương trình con?

Khi một công việc cần được thực hiện lặp đi lặp lại ở nhiều nơi trong chương trình, ta nên viết công việc đó thành một hàm. Điều này giúp chương trình rõ ràng, dễ hiểu và dễ bảo trì hơn.

Ví dụ 1: Viết chương trình in các số nguyên tố nhỏ hơn hoặc bằng n.

Ta có thể sử dụng lại hàm prime(n) đã học để kiểm tra tính nguyên tố cho mỗi số.

Python

Hàm kiểm tra số nguyên tố

```
def prime(n):
    if n < 2: return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

--- Chương trình chính ---

```
n = int(input("Nhập số tự nhiên n: "))
for k in range(1, n + 1):
    if prime(k): # Gọi hàm prime() để kiểm tra k
        print(k, end=" ")
```

Ví dụ 2: Tính tổng các số hạng dương của hai dãy số A và B.

Ta sẽ viết một hàm tongduong(danh_sach) rồi gọi nó cho cả hai dãy.

Python

```
def tongduong(danh_sach):
    S = 0
    for k in danh_sach:
        if k > 0:
            S = S + k
    return S
```

--- Chương trình chính ---

A = [0, 2, -1, 5, 10, -3]

B = [1, -10, -11, 8, 2, 0, -5]

tong_A = tongduong(A) # Gọi hàm cho dãy A

tong_B = tongduong(B) # Gọi hàm cho dãy B

print("Tổng các số dương của A:", tong_A) # Kết quả: 17

print("Tổng các số dương của B:", tong_B) # Kết quả: 11

GHI NHỚ

Sử dụng chương trình con (hàm) giúp:

- Phân chia bài toán lớn thành các bài toán nhỏ hơn.
- Tái sử dụng mã lệnh, tránh lặp lại.
- Chương trình có cấu trúc rõ ràng, dễ hiểu, dễ sửa lỗi và nâng cấp.

THỰC HÀNH

Nhiệm vụ 1: Thiết lập hàm `f_sum(A, b)` có chức năng:

- Nếu `b = 0`, tính tổng tất cả các số của danh sách A.
- Nếu `b` khác 0, chỉ tính tổng các số dương của A.

Python

```
def f_sum(A, b):
```

```
    S = 0
```

```
    for x in A:
```

```
        if b == 0:
```

```
            S = S + x
```

```
        else:
```

```
            if x > 0:
```

```
                S = S + x
```

```
    return S
```

LUYỆN TẬP

1. Thiết lập hàm `power(a, b, c)` với `a, b, c` là số nguyên. Hàm trả lại giá trị $(a+b)^c$.
2. Viết chương trình thực hiện: Nhập hai số tự nhiên từ bàn phím (cách nhau bởi dấu cách), sau đó tính và in ra tổng của chúng. Yêu cầu sử dụng hàm khi viết chương trình.

VẬN DỤNG

1. Viết chương trình nhập hai số tự nhiên (cách nhau bởi dấu phẩy), in ra ước chung lớn nhất (ƯCLN) của hai số. Yêu cầu sử dụng hàm khi viết chương trình.
2. Thiết lập hàm `change(ho_ten, c)`:
 - Nếu `c = 0`, hàm trả lại xâu `ho_ten` là chữ in hoa.
 - Nếu `c` khác 0, hàm trả lại xâu `ho_ten` là chữ in thường.
 - **Gợi ý:** Sử dụng các phương thức `s.upper()` và `s.lower()`.